CI/CD Pipeline Implementation Guide

GitHub → Jenkins/GitHub Actions → Docker → EC2

This guide covers implementing a complete CI/CD pipeline for a web application using modern DevOps tools.

Prerequisites

- AWS account with appropriate permissions
- GitHub account
- Basic knowledge of Docker, Git, and command line
- A web application to deploy (Node.js, Python, Java, etc.)

Phase 1: AWS EC2 Setup

Step 1: Launch EC2 Instance

1. **Login to AWS Console**
   - Navigate to EC2 service
   - Click "Launch Instance"
2. **Configure Instance**
   - Name: `cicd-server`
   - AMI: Ubuntu Server 22.04 LTS (Free tier eligible)
   - Instance Type: t2.micro (or t3.small for better performance)
   - Key Pair: Create new or select existing
   - Security Group: Create with following rules:
     - SSH (22) - Your IP

- HTTP (80) - 0.0.0.0/0
- HTTPS (443) - 0.0.0.0/0
- Custom TCP (8080) - 0.0.0.0/0 (for Jenkins)
- Custom TCP (3000) - 0.0.0.0/0 (for app)

3. **Launch Instance**
   - Review settings and launch
   - Note down public IP address

Step 2: Connect to EC2 Instance

```bash
# Connect via SSH
ssh -i your-key.pem ubuntu@your-ec2-public-ip

# Update system
sudo apt update && sudo apt upgrade -y
```

Step 3: Install Docker

```bash
```

```
# Install Docker
sudo apt install docker.io -y

# Start and enable Docker
sudo systemctl start docker
sudo systemctl enable docker

# Add user to docker group
sudo usermod -aG docker ubuntu

# Logout and login again, then verify
docker --version
```

Phase 2: GitHub Repository Setup

Step 4: Prepare Your Repository

1. **Create/Clone Repository**

```
bash

# Clone your existing repo or create new one
git clone https://github.com/yourusername/your-web-app.git
cd your-web-app
```

2. **Create Dockerfile**

```
dockerfile
```

```
# Example for Node.js app
FROM node:16-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

3. **Create .dockerignore**

```
node_modules
npm-debug.log
.git
.gitignore
README.md
.env
coverage
.nyc_output
```

4. **Create docker-compose.yml** (optional)

```
yaml
```

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
```

Phase 3: Option A - Jenkins Setup

Step 5: Install Jenkins on EC2

bash

```
# Install Java
sudo apt update
sudo apt install openjdk-11-jdk -y

# Add Jenkins repository
wget -q -O - https://pkg.jenkins.io/debian/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ > /etc/apt/sources.list.d/jenkins.list'

# Install Jenkins
sudo apt update
sudo apt install jenkins -y

# Start Jenkins
sudo systemctl start jenkins
sudo systemctl enable jenkins

# Get initial admin password
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Step 6: Configure Jenkins

1. **Access Jenkins**
   - Open browser: `http://your-ec2-ip:8080`
   - Enter initial admin password
   - Install suggested plugins
2. **Install Additional Plugins**
   - Go to Manage Jenkins → Manage Plugins
   - Install: Docker, GitHub, Pipeline, Blue Ocean
3. **Configure Global Tools**

- Manage Jenkins → Global Tool Configuration
- Add Docker installation
- Add Git installation

Step 7: Create Jenkins Pipeline

1. **Create New Pipeline Job**
   - New Item → Pipeline → Name: `web-app-pipeline`
2. **Configure Pipeline**
   - Pipeline Definition: Pipeline script from SCM
   - SCM: Git
   - Repository URL: Your GitHub repo
   - Branch: */main
3. **Create Jenkinsfile**

```groovy
```

```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = 'your-app'
        DOCKER_TAG = "${BUILD_NUMBER}"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                script {
                    docker.build("${DOCKER_IMAGE}:${DOCKER_TAG}")
                }
            }
        }

        stage('Test') {
            steps {
                script {
                    docker.image("${DOCKER_IMAGE}:${DOCKER_TAG}").inside {
                        sh 'npm test'
                    }
                }
            }
        }
```

```
        stage('Deploy') {
            steps {
                script {
                    // Stop existing container
                    sh 'docker stop your-app || true'
                    sh 'docker rm your-app || true'

                    // Run new container
                    sh "docker run -d --name your-app -p 3000:3000 ${DOCKER_IMAGE}:${DOCKER_TAG}"
                }
            }
        }
    }

    post {
        always {
            // Clean up
            sh 'docker image prune -f'
        }
    }
}
```

Phase 3: Option B - GitHub Actions Setup

Step 8: GitHub Actions Configuration

1. **Create Workflow Directory**

```
bash

mkdir -p .github/workflows
```

2. **Create Workflow File**

```
yaml
```

```yaml
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3

    - name: Setup Node.js
      uses: actions/setup-node@v3
      with:
        node-version: '16'
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Run tests
      run: npm test

    - name: Build application
      run: npm run build

  deploy:
    needs: test
```

```
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
    - uses: actions/checkout@v3

    - name: Deploy to EC2
      uses: appleboy/ssh-action@v0.1.5
      with:
        host: ${{ secrets.EC2_HOST }}
        username: ${{ secrets.EC2_USER }}
        key: ${{ secrets.EC2_SSH_KEY }}
        script: |
          cd /home/ubuntu/your-app
          git pull origin main
          docker build -t your-app .
          docker stop your-app || true
          docker rm your-app || true
          docker run -d --name your-app -p 3000:3000 your-app
```

Step 9: Configure GitHub Secrets

1. **Go to Repository Settings**
   - Settings → Secrets and variables → Actions
2. **Add Secrets**
   - `EC2_HOST`: Your EC2 public IP
   - `EC2_USER`: ubuntu
   - `EC2_SSH_KEY`: Your private key content

Phase 4: Deployment Setup

Step 10: Prepare EC2 for Deployment

```bash
# Create app directory
sudo mkdir -p /home/ubuntu/your-app
sudo chown ubuntu:ubuntu /home/ubuntu/your-app

# Clone repository
cd /home/ubuntu
git clone https://github.com/yourusername/your-web-app.git your-app
cd your-app

# Initial build
docker build -t your-app .
```

Step 11: Create Deployment Script

```bash
```

```
# Create deploy.sh
cat > deploy.sh << 'EOF'
#!/bin/bash

# Pull latest code
git pull origin main

# Build new image
docker build -t your-app:latest .

# Stop and remove old container
docker stop your-app || true
docker rm your-app || true

# Run new container
docker run -d --name your-app -p 3000:3000 your-app:latest

# Clean up old images
docker image prune -f

echo "Deployment completed!"
EOF

# Make executable
chmod +x deploy.sh
```

Phase 5: Advanced Configuration

Step 12: Environment Variables

1. **Create .env file**

```
bash
```

```
# .env
NODE_ENV=production
PORT=3000
DATABASE_URL=your-database-url
```

2. **Update Docker run command**

```
bash

docker run -d --name your-app -p 3000:3000 --env-file .env your-app:latest
```

Step 13: Health Checks

1. **Add health check endpoint**

```
javascript

// For Node.js/Express
app.get('/health', (req, res) => {
  res.status(200).json({ status: 'OK', timestamp: new Date().toISOString() });
});
```

2. **Update Dockerfile**

```
dockerfile

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1
```

Step 14: SSL Certificate (Optional)

```
bash
```

```
# Install Certbot
sudo apt install certbot python3-certbot-nginx -y

# Get certificate
sudo certbot certonly --standalone -d your-domain.com

# Configure reverse proxy with Nginx
sudo apt install nginx -y
```

Step 15: Monitoring and Logging

1. **Add logging to application**

```
javascript

// Example for Node.js
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.json(),
  transports: [
    new winston.transports.File({ filename: 'app.log' })
  ]
});
```

2. **Docker logging**

```
bash
```

```
# View logs
docker logs your-app

# Follow logs
docker logs -f your-app
```

Phase 6: Testing the Pipeline

Step 16: Test the Complete Pipeline

1. **Make a code change**

```
bash

# Edit a file
echo "console.log('Pipeline test');" >> app.js

# Commit and push
git add .
git commit -m "Test CI/CD pipeline"
git push origin main
```

2. **Monitor the pipeline**
   - Jenkins: Check build status at `http://your-ec2-ip:8080`
   - GitHub Actions: Check Actions tab in repository

3. **Verify deployment**

```
bash
```

```
# Check if container is running
docker ps

# Test application
curl http://your-ec2-ip:3000
```

Phase 7: Troubleshooting

Common Issues and Solutions

1. **Docker permission denied**

```
bash

sudo usermod -aG docker $USER
newgrp docker
```

2. **Port already in use**

```
bash

sudo lsof -i :3000
sudo kill -9 PID
```

3. **Container fails to start**

```
bash

docker logs your-app
docker run -it your-app sh  # Interactive debugging
```

4. **GitHub Actions SSH connection failed**
   - Verify EC2 security group allows SSH
   - Check secrets are correctly configured

- Ensure private key format is correct

Phase 8: Best Practices

Security Considerations

1. **Use secrets for sensitive data**
2. **Implement proper IAM roles**
3. **Regular security updates**
4. **Use non-root user in containers**

Performance Optimization

1. **Multi-stage Docker builds**
2. **Docker layer caching**
3. **Resource limits**
4. **Load balancing for multiple instances**

Backup Strategy

1. **Database backups**
2. **Code repository backups**
3. **Docker image registry**
4. **Configuration backups**

Conclusion

This CI/CD pipeline provides automated building, testing, and deployment of your web application. The pipeline triggers on code changes, builds a Docker image, runs tests, and deploys to EC2 automatically.

Key benefits achieved:

- Automated deployment process
- Consistent environments
- Quick rollback capabilities
- Reduced manual errors
- Faster development cycles

Remember to regularly update your dependencies, monitor your application, and implement proper backup strategies for production use.